CHAPTER

TWENTYONE

ACPI DATA TABLES AND TABLE DEFINITION LANGUAGE

There are two fundamental types of ACPI tables:

- Tables that contain AML code produced from the ACPI Source Language (ASL). These include the DSDT, any SSDTs, and sometimes OEM-specific tables (OEMx).
- Tables that contain simple data and no AML byte code. These types of tables are known as ACPI Data Tables. They include tables such as the FADT, MADT, ECDT, SRAT, etc. - essentially any table other than a DSDT or SSDT.
- The first type of table is generated using an ASL compiler and this language is specified in section 18.

The second type of table, the ACPI Data Table, is addressed by this section.

This section describes a simple language (the Table Definition Language or TDL) that can be used to generate any ACPI data table. It simplifies the table generation for platform firmware vendors and can automatically generate fields such as table lengths, subtable lengths, checksums, flag fields, etc.

21.1 Types of ACPI Data Tables

In the context of a compiler for the Table Definition Language (TDL), there are two types of ACPI Data Tables:

- ACPI tables that are "known" to the compiler. These would typically include all of the basic ACPI tables defined in the ACPI specification such as the FADT, MADT, ECDT, etc. Since these tables are fully specified (usually via the ACPI specification, but from other sources as well), the TDL compiler knows all details of these tables – including all required data types, optional or required sub-tables, etc.
- ACPI tables that are unknown to the compiler. These may include tables that are not defined in the ACPI specification such as MCFG, DBGP, etc., or simply new ACPI tables that have not yet been implemented in the compiler.

One of the goals of the ACPI Table Definition Language is to support both cases above. Most ACPI tables will be known to the compiler (and will be the easiest to specify in TDL), but the language is general enough to allow the definition of new ACPI tables that are unknown or unimplemented in the compiler.

An additional goal of TDL is to support the output of a disassembler that formats an existing table into TDL. This enables disassembler/change/compile operations.

21.2 ACPI Table Definition Language Specification

The following section defines the ACPI Table Definition Language (TDL). The grammar notation follows the same rules as the ASL source language (See Section 19.2.1). Full definition of the various data types follows the ASL grammar specification.

21.2.1 Overview of the Table Definition Language (TDL)

Most ACPI tables share the following structure (all except FACS):

- A common, 36 byte header containing the table signature, length, checksum, revision, and other data.
- A table body which contains the specific table data.

The Table Definition Language allows the definition of an ACPI table via a collection of fields. Each line of TDL source code is a field, and corresponds to a single data item in the definition of the table.

For example, the C definition of the common ACPI table header is as follows:

```
typedef struct acpi_table_header
{
    char Signature[4];
    UINT32 Length;
    UINT8 Revision;
    UINT8 Checksum;
    char OemId[6];
    char OemTableId[8];
    UINT32 OemRevision;
    char AslCompilerId[4];
    UINT32 AslCompilerRevision;
} ACPI_TABLE_HEADER;
```

In the Table Definition Language, an ACPI table header can be described as follows:

```
: "ECDT"
: 00000000
: 01
: 00
: "OEM "
: "MACHINE1"
: 00000001
: ""
: 00000000
```

Additionally and optionally, it can also be described with accompanying field names:

```
            Signature : "ECDT" [Embedded Controller Boot Resources Table]
         Table Length : 00000000
             Revision : 01
             Checksum : 00
               Oem ID : "OEM "
         Oem Table ID : "MACHINE1"
         Oem Revision : 00000001
      Asl Compiler ID : ""
Asl Compiler Revision : 00000000
```

Note

In the ACPI table header, the TableLength, Checksum, AslCompilerId, and the AslCompilerRevision fields are all output fields that are filled in automatically by the compiler during table generation. Also, the field names are output by a disassembler that formats existing tables into TDL code.

21.2.2 TDL Grammar Specification

// Root Term

DataTable :=
FieldList

// Field Terms

FieldList :=
Field | <*field fieldlist*>

Field :=
<*fielddefinition optionalfieldcomment*> | *CommentField*

FieldDefinition :=

// Fields for predefined (known) ACPI tables

<*OptionalFieldName* ':' *FieldValue*> |

// Generic data types (used for custom or undefined ACPI tables)

<'uint8' ':' *integerexpression*> // 8-bit unsigned integer
<'uint16' ':' integerexpression> // 16-bit unsigned integer
<'uint24' ':' integerexpression> // 24-bit unsigned integer
<'uint32' ':' integerexpression> // 32-bit unsigned integer
<'uint40' ':' integerexpression> // 40-bit unsigned integer
<'uint48' ':' integerexpression> // 48-bit unsigned integer
<'uint56' ':' integerexpression> // 56-bit unsigned integer
<'uint64' ':' integerexpression> // 64-bit unsigned integer
<'string' ':' *String*> // Quoted ASCII string
<'unicode' ':' *String*> // quoted ascii string -> Unicode string
<'buffer' ':' *byteconstlist*> // Raw buffer of 8-bit unsigned integers
<'guid' ':' *guid*> // In GUID format
<'label' ':' *label*> // ASCII label - unquoted string

OptionalFieldName :=
Nothing | *AsciiCharList* // Optional field name/description

FieldValue :=
IntegerExpression | *String* | *Buffer* | *Flags* | *Label*

OptionalFieldComment :=
Nothing | <'[' *asciicharlist* ']'>

CommentField :=
<'//' *asciicharlist newline*> | <'/' *asciicharlist* '/'> | <'[' asciicharlist ']'>

// Data Expressions

IntegerExpression :=
Integer | <*integerexpression integeroperator integerexpression*> | <'(' *integerexpression* ')'>

// Operators below are shown in precedence order. The precedence rules are the same as the C language. Parentheses have precedence over all operators.

IntegerOperator :=

'!' | '~' | '*' | '/' | '%' | '+' | '-' | '<<' | '>>' | '<' | '>' | '<=' | '>=' | '==' | '!=' | '&' | '^' | '|' | '&&' | '||'

// Data Types

String :=
<'"' asciicharlist '"'>

Buffer :=
ByteConstList

Guid :=
<*dwordconst* '-' *wordconst* '-' *wordconst* '-' *wordconst* '-' *const48*>

Label :=
AsciiCharList

// Data Terms

Integer := // duplicate definition - see previous chapter
ByteConst | *WordConst* | *Const24* | *DWordConst* | *Const40* | *Const48* | *Const56* | *QWordConst* | *LabelReference*

LabelReference :=
<'$' *label*>

Flags :=
OneBit | TwoBits

ByteConstList :=
ByteConst | <*byteconst* ' ' *byteconstlist*>

AsciiCharList :=
Nothing | *PrintableAsciiChar* | <*printableasciichar asciicharlist*>

// Terminals

ByteConst :=
0x00-0xFF

WordConst :=
0x0000 - 0xFFFF

Const24 :=
0x000000 - 0xFFFFFF

DWordConst :=
0x00000000 - 0xFFFFFFFF

Const40 :=
0x0000000000 - 0xFFFFFFFFFF

Const48 :=
0x000000000000 - 0xFFFFFFFFFFFF

Const56 :=
0x00000000000000 - 0xFFFFFFFFFFFFFF

QWordConst :-
0x0000000000000000 - 0xFFFFFFFFFFFFFFFF

OneBit :=
0 - 1

TwoBits :=
0 - 3

PrintableAsciiChar :=
0x20 - 0x7E

NewLine :=
'n'

21.2.3 Data Types

21.2.3.1 Integers

All integers in ACPI are unsigned. Four major types of unsigned integers are supported by the compiler: Bytes, Words, DWords and QWords. In addition, for special cases, there are some odd sized integers such as 24-bit and 56-bit. The actual required width of an integer is defined by the ACPI table. If an integer is specified that is numerically larger than the width of the target field within the input source, an error is issued by the compiler. Integers are expected by the data table compiler to be entered in hexadecimal with no "hex" prefix.

Examples:

```
[001]     Revision : 04 // Byte (8-bit)
[002]   C2 Latency : 0000 // Word (16-bit)
[004] DSDT Address : 00000001 // DWord (32-bit)
[008]      Address : 0000000000000001 // QWord (64-bit)
```

Length of non-power-of-two examples:

```
[003]     Reserved : 000000 // 24 bits
[007] Capabilities : 00000000000000 // 56 bits
```

21.2.3.2 Integer Expressions

Expressions are supported in all fields that require an integer value.

Supported operators (Standard C meanings, in precedence order):

```
() Parentheses
! Logical NOT
~ Bitwise ones compliment (NOT)
* Multiply
/ Divide
% Modulo
+ Add
- Subtract
<< Shift left
```

(continues on next page)

(continued from previous page)

```
>> Shift right
< Less than
> Greater than
<= Less than or equal
>= Greater than or equal
== Equal
!= Not Equal
& Bitwise AND
^ bitwise Exclusive OR
| Bitwise OR
&& Logical AND
|| Logical OR
```

Examples:

```
[001]   Revision : 04 \* (4 + 7) // Byte (8-bit)
[002] C2 Latency : 0032 + 8 // Word (16-bit)
```

21.2.3.3 Flags

Many ACPI tables contain flag fields. For these fields, only the individual flag bits need to be specified to the compiler. The individual bits are aggregated into a single integer of the proper size by the compiler.

Examples:

```
[002] Flags (decoded below) : 0005
                   Polarity : 1
               Trigger Mode : 1
```

In this example, only the Polarity and Trigger Mode fields need to be specified to the compiler (as either zero or one). The compiler then creates the final 16-bit Flags field for the ACPI table.

21.2.3.4 Strings

Strings must always be surrounded by quotes. The actual string that is generated by the compiler may or may not be null-terminated, depending on the table definition in the ACPI specification. For example, the OEM ID and OEM Table ID in the common ACPI table header (shown above) are fixed at six and eight characters, respectively. They are not necessarily null terminated. Most other strings, however, are of variable-length and are automatically null terminated by the compiler. If a string is specified that is too long for a fixed-length string field, an error is issued. String lengths are specified in the definition for each relevant ACPI table.

Escape sequences within a quoted string are not allowed. The backslash character '' refers to the root of the ACPI namespace.

Examples:

```
[008]         Oem Table ID : "TEMPLATE"   // Fixed length
[006] Processor UID String : "\CPU0 "     // Variable length
```

21.2.3.5 Buffers

A buffer is typically used whenever the required binary data is larger than a QWord, or the data does not fit exactly into one of the standard integer widths. Examples include UUIDs and byte data defined by the SLIT table.

Examples:

// SLIT entry

[032] Locality 0
[0A 10 16 17 18 19 1A 1B 1C 1D 1E 1F 20 21 22 23 \] 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33

// DMAR entry

[002] PCI Path : 1F 07

Each hexadecimal byte should be entered separately, separated by a space. The continuation character (backslash) may be used to continue the buffer data to more than one line.

21.2.4 Fields Set Automatically by the Compiler

There are several types of ACPI table fields that are set automatically by the compiler. This simplifies the process of ACPI table development by relieving the programmer from these tasks.

Checksums:
All ACPI table checksums are computed and inserted automatically. This includes the main checksum that appears in the standard ACPI table header, as well as any additional checksum fields such as the extended checksum that appears in the ACPI 2.0 RSDP.

Table and Subtable Lengths:
All ACPI table lengths are computed and inserted automatically. This includes the master table length that appears in the common ACPI table header, and the length of any internal subtables as applicable.

Examples:

```
[004]  Table Length : 000000F4
[001] Subtable Type : 08 <platform interrupt sources>
[001]        Length : 10
[001] Subtable Type : 01 <memory affinity>
[001]        Length : 28
```

Flags:
As described in the previous section, individual flags are aggregated automatically by the compiler and inserted into the ACPI table as the correctly sized and valued integer.

Compiler IDs:
The data table compiler automatically inserts the ID and current revision for iASL into the common ACPI table header for each table during compilation.

21.2.5 Special Fields

Reserved Fields:
All fields that are declared as Reserved by the table definition within the ACPI (or other) specification should be set to zero.

Table Revision:
This field in the common ACPI table header is often very important and defines the structure of the remaining table. The developer should take care to ensure that this value is correct and current. This field is not set automatically by the compiler. It is instead used to indicate which version of the table is being compiled.

Table Signature:
There are several table signatures within ACPI that are either different from the table name, or have unusual length:

FADT - signature is "FACP".

MADT - signature is "APIC".

RSDP - signature is "RSD PTR " (with trailing space)

21.2.6 TDL Generic Data Types

The following data types are used to construct ACPI tables that are not predefined (known) by the TDL compiler:

UINT8 Generates an 8-bit unsigned integer UINT16 Generates a 16-bit unsigned integer UINT24 Generates a 24-bit unsigned integer UINT32 Generates a 32-bit unsigned integer UINT40 Generates a 40-bit unsigned integer UINT48 Generates a 48-bit unsigned integer UINT56 Generates a 56-bit unsigned integer UINT64 Generates a 64-bit unsigned integer String Generates a null-terminated ASCII string (ASCIIZ) Unicode Generates a null terminated Unicode (UTF-16) string Buffer Generates a buffer of 8-bit unsigned integers GUID Generates an encoded GUID in a 16-byte buffer Label Generates a Label at the current location (offset) within the table. This label can be referenced within integer expressions by prepending the label with a '$' sign.

21.2.7 Defining a Known ACPI Table in TDL

It is expected that most ACPI tables that will be created via the TDL compiler are ACPI tables that are known to the compiler. This means that the compiler contains the required structure and definition of the table, as per the ACPI specification or other specification for that table.

For these known ACPI tables, specifying the data for the table involves simply defining the value for each field in the table. The compiler automatically types the data, performs range and any value checks, and generates the appropriate output.

The starting point for any of the known ACPI tables is the document that specifies the format of the table (usually the ACPI specification), or a table template file generated by an ASL compiler, or even the output of an AML disassembler. Writing the TDL code involves implementing one line of code for each data item specified in the table definition itself.

For example, the table header for an ACPI table can be defined as simply a sequence of strings and integers. The TDL compiler will format these data items into a 36-byte ACPI header:

```
: "ECDT"
: 00000000
: 01
: 00
: "OEM "
```

(continues on next page)

(continued from previous page)

```
: "MACHINE1"
: 00000001
: ""
: 00000000
```

21.2.8 Defining an Unknown or New ACPI table in TDL

For ACPI tables that are new or whose formats are otherwise unknown to the compiler, "generic" data types are introduced to allow the definition of these tables using explicit data types.

Examples of Generic Data Types:

```
     Label : StartRecord
     UINT8 : 11
    UINT16 : $EndRecord - $StartRecord // Record length
    UINT24 : 112233
    UINT32 : 11223344
    UINT56 : 11223344556677
    UINT64 : 1122334455667788

    String : "This is a string"
DevicePath : "\PciRoot(0)\Pci(0x1f,1)\Usb(0,0)"
   Unicode : "This string will be encoded to Unicode"

    Buffer : AA 01 32 4C 77
      GUID : 11223344-5566-7788-99aa-bbccddeeff00
     Label : EndRecord
```

21.2.9 Table Definition Language Examples

21.2.9.1 ECDT Disassembler Output

The output of the iASL disassembler may be used as direct input to the TDL compiler:

```
[000h 0000 4]                 Signature : "ECDT" [Embedded Controller Data Table]
[004h 0004 4]              Table Length : 0000004E
[008h 0008 1]                  Revision : 01
[009h 0009 1]                  Checksum : F4
[00Ah 0010 6]                    Oem ID : "INTEL "
[010h 0016 8]              Oem Table ID : "TEMPLATE"
[018h 0024 4]              Oem Revision : 00000001
[01Ch 0028 4]           Asl Compiler ID : "INTL"
[020h 0032 4]     Asl Compiler Revision : 20110316

[024h 0036 12]  Command/Status Register :      [Generic Address Structure]
[024h 0036 1]                  Space ID : 01   [SystemIO]
[025h 0037 1]                 Bit Width : 08
[026h 0038 1]                Bit Offset : 00
[027h 0039 1]      Encoded Access Width : 00   [Undefined/Legacy]
[028h 0040 8]                   Address : 0000000000000066
```

(continues on next page)

(continued from previous page)

```
[030h 0048 12]            Data Register :      [Generic Address Structure]
[030h 0048 1]                  Space ID : 01   [SystemIO]
[031h 0049 1]                 Bit Width : 08
[032h 0050 1]                Bit Offset : 00
[033h 0051 1]      Encoded Access Width : 00   [Undefined/Legacy]
[034h 0052 8]                   Address : 0000000000000062

[03Ch 0060 4]                       UID : 00000000
[040h 0064 1]                GPE Number : 09
[041h 0065 13]                 NamePath : "\_SB.PCI0.EC"

Raw Table Data: Length 78 (0x4E)

  0000: 45 43 44 54 4E 00 00 00 01 F4 49 4E 54 45 4C 20  ECDTN.....INTEL
  0010: 54 45 4D 50 4C 41 54 45 01 00 00 00 49 4E 54 4C  TEMPLATE....INTL
  0020: 16 03 11 20 01 08 00 00 66 00 00 00 00 00 00 00  ... ....f.......
  0030: 01 08 00 00 62 00 00 00 00 00 00 00 00 00 00 00  ....b...........
  0040: 09 5C 5F 53 42 2E 50 43 49 30 2E 45 43 00        .\_SB.PCI0.EC.
```

21.2.9.2 ECDT Definition with Field Comments

Similar to the disassembler output but simpler:

```
Signature               : "ECDT" [Embedded Controller Data Table]
Table Length            : 0000004E
Revision                : 01
Checksum                : F4
Oem ID                  : "INTEL "
Oem Table ID            : "TEMPLATE"
Oem Revision            : 00000001
Asl Compiler ID         : "INTL"
Asl Compiler Revision   : 20110316

Command/Status Register : [Generic Address Structure]
Space ID                : 01 [SystemIO]
Bit Width               : 08
Bit Offset              : 00
Encoded Access Width    : 00 [Undefined/Legacy]
Address                 : 0000000000000066

Data Register           : [Generic Address Structure]
Space ID                : 01 [SystemIO]
Bit Width               : 08
Bit Offset              : 00
Encoded Access Width    : 00 [Undefined/Legacy]
Address                 : 0000000000000062

UID                     : 00000000
GPE Number              : 09
NamePath                : "\_SB.PCI0.EC"
```

21.2.10 Minimal ECDT Definition

An example of a minimal ECDT definition with no Field Names:

```
: "ECDT" [Embedded Controller Boot Resources Table]
: 0000004E
: 01
: F4
: "INTEL "
: "TEMPLATE"
: 00000001
: "INTL"
: 20110316

: [Generic Address Structure]
: 01 [SystemIO]
: 08
: 00
: 00 [Undefined/Legacy]
: 0000000000000066

: [Generic Address Structure]
: 01 [SystemIO]
: 08
: 00
: 00 [Undefined/Legacy]
: 0000000000000062

: 00000000
: 09
: "\_SB.PCI0.EC"
```

21.2.10.1 Generic ACPI Table Definition

Tables that are not known to the TDL compiler can be defined by using the generic data types. All ACPI tables are assumed to have the common ACPI header, however:

```
Signature             : "OEMZ"
Table Length          : 00000052
Revision              : 01
Checksum              : 6C
Oem ID                : "TEST"
Oem Table ID          : "CUSTOM "
Oem Revision          : 00000001
Asl Compiler ID       : "INTL
Asl Compiler Revision : 00000001

                UINT8 : 01
                UINT8 : 08
                UINT8 : 00
                UINT8 : 00
               UINT64 : 0000000000000066
```

(continues on next page)

(continued from previous page)

```
UINT32 : 00000000
 UINT8 : 12
String : "Hello World!"
```